11: Inference for MLR

Stat 230 - Spring 2026

0.1 Overview

In this activity you will learn how to include build multiple regression models in R, including models with categorical predictors. The `cars` data set loaded below contains information on used cars for sale, including their `Price`, `Mileage`, and `Make` (a categorical variable with six levels).

```
cars <- read.csv("https://aluby.github.io/stat230/data/Cars.csv", stringsAsFactors =
```

0.2 Thinking about the data

To begin, create a scatterplot of `Price` versus `Mileage`.

```
# put your code here
```

Q1. What do you notice about the relationship between these two variables? Is a transformation necessary?

Q2. The `Make` variable is categorical with six levels: Buick, Cadillac, Chevrolet, Pontiac, SAAB, and Saturn. To include this variable in a regression model, we need use indicator (i.e., dummy) variables. How many do we need?

0.3 Fitting an MLR model

Build (fit) a multiple regression model using `log(Price)` as the response with `Mileage` and `Make` as the predictor variables. To do this, you use `+` to separate the predictor variable on the right side of the `~` in the model formula. R will automatically convert a categorical explanatory variable into a set of indicator variables.

```
# Fill in the blanks to fit the MLR model
car_lm <- lm(___ ~ ___ + ___, data = ___)

summary(car_lm)
```

Q3. Report the fitted regression equation and the R^2 value.

Q4. Which level of `Make` is the baseline? The baseline level is represented by 0's across all of the indicator variables and won't have a coefficient in the output.

Q5. What strategy did R use to create the indicator variables for `Make`? In other words, how did R decide which level of `Make` to use as the baseline?

Q6. Which of the following models best describes the one just fit: parallel lines, different slopes, or separate lines?

0.4 Plotting the fitted model

To plot a fitted regression model, we can use the `ggpredict()` and `plot()` functions in the {ggeffects} package. The `ggpredict()` function creates a data frame of predicted values from the model for each `Make` across a range of `Mileage` values. The `plot()` function then creates a plot of these predicted values.

```
# Be sure to load ggeffects!
library(ggeffects)

# First, make predictions from the model
# Fill in the blank with your model name
cars_pred <- ggpredict(___, terms = ~Mileage + Make)


# Now plot the predictions
plot(cars_pred)
```

Q7. Does this plot confirm your answer to the previous question?

Tips on plotting the model

0.5 Holding other variables at "typical" values

If you have more predictor variables in your model, then variables not specified in the `terms` argument the `ggpredict()` function are set to a specific value. Quantitative variables are set to their mean. Categorical variables are set to the mode (the level with the most observations).

0.6 Labels

If you need to adjust your axis labels or title, then you will need to add a `labs()` layer to your plot. For example,

```
plot(cars_pred) +
  labs(x = "My new x label", y = "My new y label",
       title = "My new title")
```

0.7 Fitting a model with interactions

If we believe that the association between `Price` and `Mileage` differs by `Make`, then we can include an interaction term in our model. To do this, we use * instead of + to separate the predictor variables on the right side of the ~ in the model formula. This will include both main effects and the interaction term.

```
# Fill in the blanks to fit the MLR model with interaction
car_lm2 <- lm(___ ~ ___ * ___, data = ___)

summary(car_lm2)
```

Q8. Report the fitted regression equation for Buicks and Cadillacs.

Q9. What is the R^2 value for this model? Does this model appear to fit the data better than the previous model?

0.8 Plotting the fitted model with interactions

To plot the interaction model, we again use the *ggpredict()* and *plot()* functions in the {ggeffects} package.

```
# Fill in the blank with your model name
cars_pred2 <- ggpredict(___, terms = ~Mileage + Make)

# Now plot the predictions
plot(cars_pred2)
```

0.9 Calculating extra sums of squares F-tests

Do we really need the interaction terms? To answer this question, we can use an extra sums of squares F-test to compare the model with interaction terms to the model without interaction terms.

We have already seen how to use the *anova()* function to compare two nested models. Here, we will use it again to compare our two car models.

Q10. Fill in the blanks to run the extra sums of squares F-test. What do you conclude about the interaction terms?

```
# Fill in the blanks to run the extra sums of squares F-test
anova(___, ___)
```

0.9.1 Using only the full model

We can also calculate the extra sums of squares F-test using only the full model. To do this, we need to extract the SSE and degrees of freedom for both models from the ANOVA table for the full model.

Q11. Run the below code and verify that the df, sums of squares, mean squares, F value, and p-value match what you got from the previous *anova()* command.

```
anova(car_lm2)
```

0.10 Function quick reference

The following table summarizes the functions we learned today:

Function	Purpose
`lm()`	Fit a linear model
`summary()`	Display detailed results from a fitted model
`ggpredict()`	Create a data frame of predicted values from a fitted model
`plot()`	Create a plot of predicted values from a fitted model
`anova()`	Create an ANOVA table for a fitted model or compare two nested models